ISEL | DEI | AMD / MLDM

Machine Learning and Data Mining

1. Context and objectives

This term-project (TeP) is based on real and recent hospital activity data obtained from Portugal's National Health Service (SNS) Transparency Portal. The dataset, *Morbilidade e Mortalidade Hospitalar*, covers the period from January 2021 until recently and provides monthly records of inpatient admissions, ambulatory episodes, inpatient days, and deaths; data is organized by reference month, hospital institution, diagnosis chapter, age group, and sex.

Working with real-world data introduces additional challenges, including data quality issues, such as inconsistencies, and ambiguities. These challenges require special attention during the entire process, from data integration to predictive modelling, evaluation, and deployment.

data-source: https://transparencia.sns.gov.pt/explore/assets/morbilidade_mortalidade_hospit/

1.1. Observation-Unit and Prediction-Time Information

The **observation-unit** is defined by five attributes:

- `(period, institution, diagnosis, age_group, sex)`,

establishing the monthly data-grain of the **analytical-dataset**.

Each observation-unit identifies aggregated hospital activity, rather than an individual patient or hospitalization and is described by the:

- number of inpatient admissions, ambulatory episodes, inpatient days, and deaths

Repeated source records must be consolidated to ensure one record per observation-unit.

The five observation-unit attributes may be used to construct modelling features. The hospital activity, recorded for the month being predicted, is used to construct the targets, not the input features.

The **modelling-datasets** are derived from the analytical-dataset according to each prediction problem and the selected feature representation.

1.2. Prediction Problems

The TeP addresses **three prediction problems** that share the same observation-unit but differ in their learning tasks and targets:

- **death occurrence | binary classification** | predict whether at least one death occurs during the month. The *target*: `has_death` (`0`: no deaths; `1`: one or more deaths).
- **inpatient activity level | multiclass classification** | predict the level of inpatient admissions relative to total inpatient (`I`) admissions and the ambulatory (`A`) episodes. The *target*: `inpatient_level`, obtained by discretizing `I / (I + A)` into `low`, `medium`, and `high`, using the configurable intervals stored in the database.

- **inpatient days | regression** | predict the total number of inpatient days recorded during the month. The *target*: `num_inpatient_day`.

Each prediction problem requires an appropriate modelling-dataset, learning algorithms, and evaluation procedures. The goal is to investigate and compare models for all three problems.

Final Report Requirements:

- **R01** - identify one concrete prediction-time information leakage risk and explain how it was prevented in the developed solution.

---

2. Data Sources and Preparation

The SNS data sources comprise an initial dataset (A) and additional records (B). The data preparation follows a sequence of integration into the relational database, consolidation of repeated observation-units, and construction and validation of the analytical-dataset for the three prediction problems.

2.1. Provided Resources

The SNS data is provided in two datasets:

- `dataset_A` | records from 2021 to 2024, used in the *selected-solutions* (provided) notebook to explore data integration, consolidation, and analytical-dataset construction.
- `dataset_B` | additional, more recent records to be integrated and consolidated, extending the analytical-dataset constructed from `dataset_A`.

The provided resources also include:

- PostgreSQL database initialization scripts
- reference data, and
- a *selected-solutions* notebook, with selected support and `TODO` items

---

2.2. Relational Model Analysis

The provided PostgreSQL database includes a relational model that must be analysed to reconstruct an **Entity–Relationship (ER) diagram**, identifying:

- the entities
- their attributes, including primary keys
- the relationships between entities
- the cardinality of each relationship
- the optionality of each relationship

2.3. Data Integration and Consolidation

The data integration goal is to bring datasets A and B together in the relational database, preserving the original data and supporting the construction of a consolidated analytical-dataset.

Unlike datasets specifically prepared for academic exercises, these real-world data have not been cleaned or simplified, requiring careful analysis and decisions regarding data consistency.

The analysis of the SNS data revealed that *some records sharing the same observation-unit contain different hospital activity values*. Clarification was requested (via email) from the SNS data provider, but no response has been received to date.

The **consolidation** involves identifying records that share the same observation-unit and selecting **exactly one** original record per observation-unit.

The conflicting records are resolved using a deterministic *worst-case selection heuristic*, prioritizing, in descending order, the number of deaths, inpatient days, inpatient admissions, and ambulatory episodes; a stable record identifier resolves any remaining ties.

The *worst-case* rationale prioritizes higher reported mortality and, subsequently, greater hospital resource utilization, avoiding the systematic selection of lower reported values.

This **worst-case criterion is an operational selection** decision, not a clinically validated assessment of severity. This approach preserves an original record and ensures reproducibility, but does not guarantee that the selected values represent the actual hospital activity.

The reference tables populated by `_mLDm_02_sns_data_db.sql` provide the complete and authoritative reference values for institutions, regions, and diagnoses used in datasets A and B.

Any differences between these reference tables' values and the source records' values and must be identified and **explicitly reconciled (corrected) before integration**.

The data corrections must be documented in `03_output/sns_integration_check.csv`, identifying the source dataset, affected attribute, original value, and adopted reference value. A single entry may represent multiple source records sharing the same correction.

The original input files must remain unchanged. All corrections must be reproducible via the integration code. The records without a justified correspondence must not be silently discarded.

The same consolidation criterion must be applied consistently to datasets A and B.

2.4. Analytical-Dataset

The **analytical-dataset** must be constructed from the integrated and consolidated records of `dataset_A` and `dataset_B`, containing exactly one record per observation-unit.

Each record must contain the five observation-unit attributes and the three prediction targets:

- `inpatient_level`
- `num_inpatient_day`
- `has_death`

The analytical-dataset serves as the common source for constructing the three modelling-datasets and must be exported to:

- `03_output/sns_analytical_dataset.csv`

The analytical-dataset must be validated by verifying:

- uniqueness of observation-units
- correct derivation and validity of the three targets

- absence of unintended loss of observation-units during integration and consolidation

Final Report Requirements:

- **R02** - include an Entity–Relationship (ER) diagram built from the provided relational model, identifying entities, attributes, identifiers, relationships, cardinalities and optionality.
- **R03** - quantify original records, distinct observation-units, repetitions and conflicts in datasets A and B, and characterize the target distributions.
- **R04** - assess the impact of the *worst-case* consolidation criterion on one prediction target, comparing the adopted rule with an alternative deterministic selection rule.

3. Predictive Modelling

The three prediction problems require the construction of appropriate modelling-datasets and the development of predictive models. The modelling process involves selecting and transforming attributes, choosing learning algorithms, and fitting models to the observations.

3.1. Modelling-Datasets

Each prediction problem requires a **modelling-dataset** derived from the common analytical-dataset. The modelling-datasets must identify the corresponding target and provide an appropriate feature representation, obtained through attribute selection and transformation.

Different feature representations may be investigated, including:

- categorical and numerical feature representations
- temporal feature representations
- features derived from the observation-unit attributes
- and other representations considered appropriate

The feature construction must respect the prediction-time constraints previously established; e.g., hospital activity values from the month being predicted cannot be used as input features.

3.2. Single-Feature Classifier (1R)

The **One-Rule (1R)** classifier algorithm is a particular case of the *single-feature classifiers* (as introduced in a previous module of theory).

Each candidate classifier uses a single categorical feature and constructs a fixed partition of **K blocks**, where K is the number of distinct values (for that feature) observed in the training data. Each block corresponds to one feature value and predicts its majority target class. Unlike more general single-feature classifiers, 1R does not search for alternative partitions of feature values.

For each categorical feature:

- construct the contingency matrix from the training data
- identify the majority target class for each feature value
- calculate the error for each block as the row total minus the majority-class count
- sum the block errors to obtain the candidate classifier's total training error

The **1R algorithm** selects the candidate classifier with the lowest total training error.

The 1R algorithm **must** be implemented as a `OneRClassifier` Python class. The implementation **must** be based on **contingency matrices**, using DuckDB `PIVOT`, with:

- feature values as rows
- target classes as columns, and
- their frequencies as entries

The implementation of `OneRClassifier` is required, with the following characteristics:

- mandatory use of DuckDB `PIVOT` to construct the contingency matrices
- deterministic resolution of ties
- for previously unseen feature values, predict the global training majority class

The provided *selected-solutions* notebook includes an illustrative example of constructing and processing a contingency matrix using DuckDB `PIVOT` and Python.

The resulting classifier must:

- be applied to both classification problems
- apply the same evaluation procedures as all *other predictive models* (cf., next section)

3.3. Other Predictive Models

In addition to the mandatory **1R implementation**, three learning techniques must be **explored**:

- **Decision Trees**
- **k-Nearest Neighbors (kNN)**
- **Random Forests**

These techniques must be applied to the three prediction problems, using their classification or regression variants as appropriate.

Additional learning techniques may also be freely selected and investigated, allowing the exploration of different modelling approaches and feature representations.

All models must:

- support reproducible training and prediction
- have their configurations documented

Final Report Requirements:

- **R05** - compare and justify the feature representations for the three prediction problems.
- **R06** - analyse 1R results for both problems, using contingency matrices to explain the selected features, classification errors, and differences to the majority-class baselines.

---

4. Experimental Evaluation

The three prediction problems require experimental evaluation to assess model performance, compare alternative configurations, and support model selection. The evaluation procedures

must be reproducible, consistent across competing models, and appropriate to the temporal nature of the hospital activity data.

4.1. Temporal Evaluation

The evaluation must respect the chronological order of observations, ensuring that models are trained exclusively on information available before the periods used for testing.

An **expanding-window cross-validation** approach must be adopted, progressively extending the training period and testing predictions on subsequent observations.

The evaluation procedure must:

- define the temporal training and testing partitions
- use consistent partitions when comparing models for the same prediction problem
- fit all data-dependent transformations exclusively on the corresponding training partitions
- prevent information leakage between training and testing

The temporal evaluation procedure must be documented and reproducible.

4.2. Model Evaluation and Selection

Each prediction problem must include a simple **baseline model** reference against which alternative predictive models are compared.

For classification, the majority-class predictor provides a baseline. For regression, a constant predictor based on the mean target value in the training data provides an appropriate baseline.

The evaluation must include:

- **binary and multiclass** | accuracy, balanced-accuracy, and macro-averaged F1-score
- **regression** | mean-absolute-error (MAE) and root-mean-squared-error (RMSE)

For classification, examine whether accuracy masks poor performance on less frequent classes, using balanced accuracy and macro-F1. For regression, compare MAE and RMSE to assess the influence of large errors, e.g., for observations with high values of `num_inpatient_day`.

The experimental results must support the **comparison and selection** of models, considering:

- baseline performance and the required learning techniques
- alternative feature representations and model configurations
- performance and its variation across temporal testing partitions

The **selected model for each prediction problem** must be justified by experimental evidence, considering performance differences and potential limitations.

Final Report Requirements:

- **R07** - characterize and justify the temporal training and testing partitions.
- **R08** - compare the investigated models against their baselines, identifying relevant differences between evaluation metrics.

- **R09** - analyse performance variation across testing periods and justify the selected model for each prediction problem.

---

5. Model Deployment

The model deployment involves using the selected models to predict new observations, independently of the training and experimental evaluation processes.

The **deployment solution** must reproduce the required feature transformations and generate predictions for all three prediction problems.

5.1. Final Models and Persistence

After completing experimental evaluation and model selection, the selected model for each prediction problem must be **retrained** using **all available observations** (i.e., from all consolidated data). This is the **final model** and is the one that must be used for **deployment**.

The three final models, with all preprocessing components, must be persisted in a single file:

- `03_output/sns_deployed_models.joblib`

This file must support standalone prediction **without retraining or execution of notebooks**.

5.2. Prediction Application

A standalone prediction application (`02_deployed_predictors.py`) must process an input CSV file (`_mLDm_00_sns_data_C.csv`) containing new observations, described by the five observation-unit attributes:

- `period`
- `institution`
- `diagnosis`
- `age_group`
- `sex`

The application (`02_deployed_predictors.py`) must:

- load the persisted models and preprocessing components (`sns_deployed_models.joblib`)
- validate the input `_mLDm_00_sns_data_C.csv`) and apply the required feature transformations
- predict the three targets - `inpatient_level`, `num_inpatient_day`, `has_death`
- save the predictions to a CSV file (`_mLDm_00_sns_data_C_prediction.csv`), preserving the five input attributes and one output record per input record, preserving the original order

---

Final Report Requirements:

- **R10** - demonstrate successful standalone execution using persisted models, verifying input/output consistency, absence of retraining, and appropriate handling of previously

unseen feature values.

6. Deliverables and Submission

The TeP follows the standard MLDM workspace structure, with four directories: `01_input`, `02_code`, `03_output`, and `04_report`. The submission ZIP includes only the required artefacts, preserving their relative paths within the workspace.

6.1. Provided Materials (must remain unchanged)

Data and Reference Files | `01_input`

- `_mLDm_00_sns_data_A.csv`, data for integration with provided notebook assistance
- `_mLDm_00_sns_data_B.csv`, additional data to complete the analytical-dataset
- `_mLDm_00_sns_data_C.csv`, new observations reserved **exclusively** for deployment
- `_mLDm_01_sns_schema.sql`, PostgreSQL relational schema
- `_mLDm_02_sns_data_db.sql`, reference data for institutions, regions, and diagnoses
- `_mLDm_xx_submission_requirements_rev01.txt`, submission requirements

Support Code | `02_code`

- `_mLDm_00_sns_db_boot.py`, database initialization script
- `_mLDm_00_sns_selected_solutions.ipynb`, selected solutions and completion tasks
- `_mLDm_xx_build_submission_zip_rev01.py`, submission script

Note: the dataset C must not be used for model training, evaluation, or selection.

6.2. Required Deliverables

To be developed and made available for submission or subsequent TeP assessment.

Code and Experiments | `02_code`:

- `00_sns_selected_solutions.ipynb`, completed version of the provided notebook, with all `TODO` tasks resolved, regardless of whether the notebook is used in the final solution
- `01_sns_build_modelling_datasets.ipynb`, to integrate and consolidate datasets A and B, construct the analytical-dataset, generate and save the modelling-datasets
- `02_deployed_predictors.py`, the standalone application to predict new observations
- *other code* (or notebooks) required for model construction, evaluation, and deployment

The organization and naming of *other code* files are freely chosen and must be explained in the final report.

The Orange DM may also be used for experimentation. Any developed workflows (`.ows` files) must be saved in `02_code` and referenced in the report, briefly explaining their purpose.

Generated Results | `03_output`:

- `sns_analytical_dataset.csv`, complete analytical-dataset constructed from A and B, required locally (but excluded from the submission file)
- modelling-datasets used in the experiments, saved locally for inspection during the project discussion, with freely chosen filenames (but excluded from the submission file)
- experimental results and other generated files required by the solution (but excluded from the submission file)
- `sns_integration_check.csv`, source-data corrections (included in submission file)
- `sns_deployed_models.joblib`, the three final models and required preprocessing components (included in submission file)
- `_mLDm_00_sns_data_C_prediction.csv`, predictions generated by the standalone application (included in submission file)

The `_mLDm_00_sns_data_C_prediction.csv` **must**:

- preserve the five input attributes (`period`, `institution`, `diagnosis`, `age_group`, `sex`), **in their original order** (i.e., as presented in `_mLDm_00_sns_data_C.csv`)
- append the three predicted targets (`inpatient_level`, `num_inpatient_day`, `has_death`), **in this order**
- preserve the number and order of input observations, with exactly one prediction per input observation

Documentation | `04_report`:

- `report.pdf`, final report
- `README.txt`, instructions for reproducing and executing the solution

---

6.3. Final Report

The final report must be a .pdf with a **maximum of 10 pages**, excluding the cover page.

The cover page must identify the course and curricular unit (AMD \ MLDM), the group number, the name and number of each group member.

The report must follow the five sections of the handout (sections 1–5), addressing the **ten reporting requirements (R01–R10)** in the same order.

Each requirement must be explicitly identified and answered via concise, evidence-based justifications, supported by tables or figures whenever possible.

The report must also:

- include the required Entity–Relationship (ER) diagram
- identify the code files relevant to the reported results

Important: emphasis must be on **results, interpretation, and justification of decisions**, rather than reproducing procedures already described in the handout.

6.4. Reproducibility (and `README.txt`)

The `04_report/README.txt` must provide sufficient instructions to reproduce and execute the developed solution, including:

- environment requirements and dependencies
- database initialization and preparation
- execution order for data preparation, model training, and result generation
- execution of `02_deployed_predictors.py`
- identification of the main generated files

All files in `03_output` must be reproducible from `01_input` and `02_code` without undocumented manual operations.

6.5. Submission Procedure

The submission, of group number `NN`, must be prepared via:

- `python3 _mLDm_xx_build_submission_zip_rev01.py NN`

The script verifies the required artefacts and generates the submission file in folder:

- `mLDm_MoP_workspace`

The verification results are recorded in:

- `04_report/submission_check.txt`.

Note: only the generated ZIP file must be submitted.